

The Mobile Platform Handbook

An agent-first engineering system for Expo / React Native teams

Skeleton version 0.1.0 · Expo SDK 56 · June 2026

Prepared for the engineering organization. This handbook explains the philosophy, research, architecture, team operating model, AI harness and CI/CD system embodied in the `expo-skeleton` repository — and the six-month roadmap for adopting the agent-first way of working.

Contents

1. **Executive summary**
2. **Part I — Philosophy & research:** why "code is a liability" and what harness engineering means
3. **Part II — The stack:** every choice, pinned and justified
4. **Part III — Repository architecture:** layers, boundaries, and the reference feature
5. **Part IV — Team operating model:** roles, lanes, and conflict-free parallel work
6. **Part V — The AI harness:** AGENTS.md, the docs tree, personas, commands, MCP
7. **Part VI — CI/CD:** quality gates, AI review, ship reports, Notion, E2E, EAS delivery
8. **Part VII — The six-month adoption roadmap**
9. **Part VIII — Governance, security & failure modes**
10. **Appendix:** setup checklist, cheatsheet, sources

1 · Executive summary

This skeleton is not a starter template; it is an **operating system for a software team** in which most code is written by AI agents directed by humans — engineers, product managers, designers and testers alike.

The repository ships four things:

A production-grade Expo application skeleton. Expo SDK 56 (React Native 0.85, React 19.2, New Architecture, Hermes v1), TypeScript strict, expo-router, Zustand 5 + Zod 4, NativeWind 4, Reanimated 4, Jest + React Native Testing Library — with a complete, tested reference feature ([tasks](#)) demonstrating every sanctioned pattern.

An enforced architecture. Three layers ([app](#) → [features](#) → [shared](#)) with module boundaries checked by ESLint, feature public APIs, schema-validated edges, and a structure where one feature equals one folder equals one owner — the property that makes parallel human/agent work conflict-free.

An AI harness. A canonical [AGENTS.md](#) operating manual (the open, Linux-Foundation-stewarded standard read by Claude Code, Cursor, Codex, Copilot, Gemini and others), a cross-linked docs tree whose integrity is enforced in CI like a failing test, reviewer personas that grade every PR before a human sees it, Claude Code slash-commands for the team's rituals, and MCP integrations: **Argent** (Software Mansion's agentic toolkit that lets agents drive the iOS Simulator/Android emulator — tap, type, screenshot, read logs, profile) and **Notion**.

A CI/CD system. Quality gates (format, lint+boundaries, types, tests, docs integrity), AI persona review on every PR, an automated **ship report**: on every push to [main](#) , Claude Code reads the actual diff, writes an engineering report rendered to **PDF**, and publishes a product-friendly **"What's new" page to Notion** via MCP; deterministic Maestro E2E on simulator builds; nightly **agentic QA** where Claude + Argent walks the critical user journeys like a tester; and EAS delivery with fingerprint-aware OTA updates, PR preview QR codes, and tag-driven store releases.

The guiding principle, borrowed from the team at OpenAI Frontier that shipped one million agent-written lines in six months: **the highest-leverage engineering work is writing things down** — and then wiring what you wrote into machinery that injects it into every agent session and rejects output that violates it.

2 · Part I — Philosophy & research

2.1 Code is a liability

Every human software organization was tooled around one assumption: code is the most expensive thing to produce, so it must be protected — synchronous reviews, defensive ownership, "elbows out" toward outside contributors. That assumption broke. With frontier coding agents, code is cheap to generate and parallelizable almost arbitrarily; what is expensive now is **validation** — knowing the generated code is correct, on-architecture, and on-taste.

The consequence is an inversion: instead of spending organizational effort producing code, you spend it building the system that makes *bad code impossible to merge* and *good context impossible to miss*. The codebase's real asset is not the code; it is the accumulated, machine-readable judgment about what good looks like.

2.2 Harness engineering — the case study behind this skeleton

The design here adapts the publicly described experiment by Ryan Lopopolo's team at OpenAI Frontier (the "1M lines, zero typed by humans" project), whose mechanics translate directly to any stack:

- **The repo is the agent's brain.** Everything the agent must know lives in the repository where it can grep it: architecture docs, conventions, design decisions (ADRs), implementation plans, product PRDs, critical user journeys, persona definitions, even a running "quality score" of code health. Knowledge buried in Slack or Google Docs does not exist for an agent.
- **agents.md points, the model pulls.** Context windows are scarce. The root manual is a *map*, not an encyclopedia — it tells the agent what resources exist, their shape, and when to read them, then lets the model's reasoning decide what to pull into context.
- **Taste becomes failing tests.** Their canonical example: the build fails if user-facing strings use straight quotes instead of typographic ones. The point is not typography; it is that *any* recurring human correction can be encoded so the correction never has to be given again. Our `docs-lint` implements exactly this class of check (link integrity, orphaned docs, AGENTS.md map completeness, and yes — curly apostrophes in UI copy) as a template to extend.
- **Feedback is given at most twice.** When an agent (or human) errs, the job is to stop, diagnose what context was missing, and make the repository itself provide it next time — a lint rule, a test, a doc. Leverage compounds: every encoded correction applies to every future change.
- **Architecture limits where code can plausibly go.** Their "spaghetti scheduler in the front-end JavaScript" failure — an entire feature trashed and reverted — happened *before* boundaries were encoded. The fix was structural: hard package layering so a bowl of mud cannot form, plus the **painted door** norm (designers ship real UI over a no-op backend, instrumented, to measure demand without infrastructure risk).
- **Close the loop.** An agent must be able to observe its own work end-to-end: run the app, click the buttons, read the logs, attest the acceptance criteria. "Vibed something up but can't attest it solves the problem" is the failure mode; closed verification loops are the cure. This is precisely the gap **Argent** fills for mobile.
- **Roles equalize, not disappear.** PM, designer, engineer and QA all operate the same agent against the same repo; each contributes their lens. Their PM shipped a feature from a markdown PRD to a customer rollout within a week without engineer hand-holding — because the harness made it safe.

The arc they reported: month one was *ten times slower* than typing the code by hand; by month six, a 60-hour agent session completed a refactor estimated at three engineer-weeks with two steering prompts. Slow is smooth; smooth becomes fast.

2.3 Why now (state of the ecosystem, June 2026)

The research pass for this skeleton confirmed the ecosystem has converged on this way of working:

- **AGENTS.md is the standard** — an open format stewarded by the Agentic AI Foundation under the Linux Foundation, used by 60k+ open-source projects and read natively by Codex, Cursor, Copilot, Gemini CLI, Devin, Jules, Zed and more; Claude Code consumes it via a one-line `CLAUDE.md` import, which is exactly how this repo wires it.
- **Expo itself went agent-first.** SDK 56's `create-expo-app` scaffolds `AGENTS.md`, `CLAUDE.md` and `.claude/settings.json` by default and Expo publishes official agent **skills** (`expo/skills`) for Claude Code and other tools. The platform expects your agents to be first-class operators.
- **Claude Code runs your CI rituals.** The official `claude-code-action` runs Claude Code inside GitHub Actions with your team's existing subscription via an OAuth token (`claude setup-token`) instead of metered API keys, supports arbitrary MCP servers, and is built for exactly the review/report automations this repo ships.

- **Agents can finally drive the phone.** Software Mansion's **Argent** (Apache-2.0 core, v0.5.x) exposes the iOS Simulator *and* Android emulator to any MCP-capable agent: build/launch apps, tap/swipe/type via the accessibility tree, deep-link, read console and network logs, capture crash reports, and run React + native profiling sessions it can re-query. It deliberately bypasses XCUITest for speed and works out of the box with React Native. This is "Playwright for mobile," plus a diagnostic suite.
- **Deterministic E2E stayed boring on purpose.** Maestro remains the YAML-flow standard for mobile E2E and is the official example in Expo's EAS Workflows docs; we use it as the *gate* and Argent as the *explorer* — determinism for regressions, agency for discovery.

3 · Part II — The stack

Every version below is the exact resolution of the official SDK 56 template plus pinned companions, verified installable and test-green at scaffold time.

LAYER	CHOICE	VERSION	WHY THIS, NOT SOMETHING ELSE
Runtime	Expo SDK	~56.0.9	RN 0.85 + React 19.2; New Architecture mandatory since SDK 55; Hermes v1 default (faster start, less memory); precompiled modules cut iOS builds ~16%; AI-ready scaffolding
Language	TypeScript	~6.0.3	SDK 56 default; <code>strict</code> + <code>noUncheckedIndexedAccess</code> — agents thrive on honest types
Navigation	expo-router	~56.2.9	File-system routes = greppable navigation; typed routes; forked from react-navigation in SDK 56 — <code>@react-navigation/*</code> imports are banned by lint
State	Zustand	^5.0.8	Minimal API agents can't misuse; selector subscriptions; persist middleware with validated rehydration
Validation	Zod	^4.1.8	Single source of domain types (<code>z.infer</code>); runtime safety at every edge; v4 is faster and slimmer than v3
Styling	NativeWind	^4.2.1 (Tailwind 3.4)	Tokens in one file; enormous training-data affordance for agents; deterministic class order via Prettier plugin. v5/Tailwind-4 is pre-release — adopt later by ADR
Animation	Reanimated	4.3.1 (+ worklets 0.8.3)	New-Architecture-native; UI-thread worklets; Babel wiring automatic via <code>babel-preset-expo</code> — a classic agent landmine, documented
Typography	@expo-google-fonts/inter + expo-font	^0.4.2	Inter 400–700 loaded at the root layout, mapped to Tailwind font tokens
Persistence	AsyncStorage	2.2.0	Default key-value store; unencrypted — never tokens/PII (SecureStore when auth lands); Zod-validated on rehydrate
Unit/ component tests	Jest + jest-expo + RNTL	^29.7 / ~56.0.4 / ^13.3.3	jest-expo mocks the native SDK; RNTL 13 (react-test-renderer is dead with React 19); 60% line floor to start
E2E	Maestro	latest CLI	YAML flows over testIDs; runs in GH Actions and natively in EAS Workflows
Agentic QA	Argent	@swmansion/ argent (v0.5.x)	MCP server + skills; simulator/emulator control, logs, network, profiling
Quality tooling	ESLint 9 (flat) + eslint-config-expo ~10 + eslint-plugin-boundaries ^5 + Prettier 3	—	Boundaries are <i>errors</i> , <code>--max-warnings 0</code> ; Prettier owns formatting; tailwind class sorting
Commit hygiene	husky + lint-staged + commitlint	^9 / ^16 / ^19	Conventional Commits feed every downstream automation
Delivery	EAS Build / Update / Workflows	CLI ≥16	Fingerprint runtime policy: JS-only → OTA in minutes; native change → build, decided automatically

Deliberate omissions (each pre-approved as a future ADR when the need is real): TanStack Query (when the first real API lands), FlashList (when a long list exists), MMKV/SecureStore (when auth lands), Sentry (when there are real users), dark mode token pass, NativeWind v5/Tailwind v4 migration.

4 · Part III — Repository architecture

4.1 The layout

```
EXPO-SKELETON/
├─ AGENTS.md           ─ agent operating manual (canonical)
├─ CLAUDE.md           ─ "@AGENTS.md" (import)
├─ app.config.ts        ─ typed Expo config; variants dev/preview/prod
├─ eas.json             ─ build profiles + channels
├─ src/
│ └─ app/               ─ expo-router routes – wiring ONLY
│   └─ features/<name>/ ─ vertical slices: ui/ model/ lib/ __tests__/ index.ts
│     └─ shared/         ─ ui kit · theme · lib (env, cn, id) · testing
├─ docs/               ─ the knowledge tree (architecture, conventions,
│   ...                ADRs, exec-plans, PRDs, personas, quality, runbooks)
├─ .claude/             ─ commands (slash) + subagents; settings & permissions
├─ .cursor/rules/ · .github/copilot-instructions.md ─ pointers to AGENTS.md
├─ .mcp.json            ─ Argent + Notion MCP servers
├─ .maestro/flows/      ─ deterministic E2E flows (one per CUJ)
├─ .github/workflows/   ─ 7 workflows (Part VI)
├─ .eas/workflows/      ─ EAS-native E2E alternative
└─ scripts/             ─ docs-lint · new-feature · ship-report-fallback
```

4.2 Boundaries that cannot be argued with

The dependency matrix — `app` — `features` — `shared`, features isolated from each other, features consumable only through `index.ts` — is enforced by `eslint-plugin-boundaries` as **errors**. The error message itself links the doc explaining the fix. Three properties fall out:

1. **Conflict-free parallelism.** Two squads (or twenty agents) in two features cannot collide; CODEOWNERS maps folders to owners 1:1.
2. **Agent containment.** An agent told to build in `features/checkout` *physically cannot* leak logic into `features/profile` or `src/app` — the build fails. This is the structural answer to the spaghetti failure mode.
3. **Cheap deletion.** A feature is one folder plus one route line. Painted-door experiments die cleanly.

4.3 The reference feature

`src/features/tasks/` is a complete worked example mapped to a real PRD (`docs/product/prds/0001-task-capture.md`) and a CUJ with a Maestro flow: Zod schema (`model/schema.ts`) as the single source of domain types; a persisted Zustand store whose `merge` validates rehydrated JSON so corrupt storage resets instead of crashing; action results as `{ ok } | { ok: false, error }`; selector exports; NativeWind UI composed from shared primitives (`AppText`, `Button`, `Screen`, `TextField`); Reanimated enter/exit list animations; feature-prefixed `testID`s shared by RNTL, Maestro and Argent; unit + component tests including the corrupt-storage path. New features are scaffolded into the same shape by `npm run new:feature`.

5 · Part IV — Team operating model

5.1 Everyone operates the same agent

ROLE	OWNS	TYPICAL LOOP	WRITE ACCESS LANE
Product (PO/PM)	Problems, PRDs, acceptance, "What's new"	Write PRD from template → <code>/plan-feature</code> → review plan → <code>/implement-prd</code> → accept against Proof section	<code>docs/product/**</code> ; feature <code>ui/</code> + <code>model/</code> via agent; never <code>shared/</code> or configs
Engineer	The harness itself	Triage & schedule agent work; review verdicts; encode every repeated correction ; own <code>shared/</code> , configs, CI	everywhere
Designer	Tokens, primitives, painted doors	Tune <code>tailwind.config.js</code> + <code>shared/ui</code> variants; ship painted-door UI with no-op backend + instrumentation	<code>tailwind.config.js</code> , <code>src/shared/ui</code> , feature <code>ui/</code>
QA / Tester	CUJs, flows, nightly agentic QA	Keep <code>critical-user-journeys.md</code> true; one Maestro flow per CUJ; triage nightly Argent reports into issues	<code>.maestro/**</code> , <code>docs/quality/**</code>
Eng lead	Throughput of the system	Watch the metrics (§8.3); run the weekly harness retro; arbitrate ADRs	everywhere

The lanes mirror the OpenAI norm: lanes are *conventions plus CODEOWNERS review*, while the hard floor (boundaries, tests, personas) is enforced by machinery — so a PM "coloring outside the lane" is caught by review, and a PM breaking architecture is caught by the build.

5.2 The rituals

- **Weekly planning (30 min)**: approve PRDs. A PRD is approved only if its acceptance criteria are testable as written.
- **Demo Friday**: PRs demo via their Proof-of-work artifacts (recordings, Argent transcripts) — not via laptops passed around.
- **Harness retro (15 min, weekly)**: read the week's `HARNESS`: proposals from AI reviews and human reviews; pick 1–3 to encode; assign. This meeting is the flywheel — guard it.
- **Quality triage (15 min)**: QA walks the latest agentic-QA report; P1s become issues; stale `quality-score.md` lines get pruned.

5.3 Conflict-free collaboration mechanics

Trunk-based development with branches living under two days; squash merges whose titles are Conventional Commits (commitlint-enforced) because every downstream automation reads them; lockfile conflicts resolved by "take main's, re-run `npm install`" (`.gitattributes merge=ours` assists); native folders never committed (CNG) — deleting an entire conflict class; and the architectural rule that cross-cutting changes (`shared/`, configs) merge fast and are announced, while feature work never crosses folders.

6 · Part V — The AI harness

6.1 AGENTS.md and its satellites

`AGENTS.md` (~150 lines — a *map*, deliberately small) defines: the operating loop (ground → plan → implement → verify → encode feedback), the command palette, the docs map ("read X when Y"), the hard rules, PR proof-of-work expectations, and the blocked-protocol ("stop after two failed attempts; ask a concrete question; encode the answer"). `CLAUDE.md` is one line — `@AGENTS.md`. `.cursor/rules/core.mdc` and `.github/copilot-instructions.md` restate the essentials and defer to `AGENTS.md`. One source of truth, every tool.

6.2 The docs tree is load-bearing

`docs/` is structured for retrieval: architecture (overview, boundaries, state, navigation, styling), ADRs (four are pre-written, including the harness decision itself), exec plans (the durable log of how things were actually built — never deleted), conventions (code style, testing, git, design system, environments), product (vision + PRD pipeline with a shipped reference PRD), quality (CUJs + the living quality-score file agents append to), personas, runbooks (setup, release, agentic QA), and a glossary.

What makes it *load-bearing* rather than aspirational is `scripts/docs-lint.mjs` in CI: broken relative links fail the build; any doc not linked from the tree fails the build (no orphan knowledge); `AGENTS.md` must mention every docs section (the map cannot rot); and the typography taste rule fails straight apostrophes in UI copy. Docs break the build exactly like tests — so they stay true, so agents can trust them, so humans bother writing them.

6.3 Personas: taste as a reviewer matrix

Five persona files — frontend-architect, ux-reviewer, security-engineer, performance-engineer, qa-engineer — each defining what it rejects, in a fixed finding format (`[P1|P2|P3] file:line — issue — fix`), with a mandatory `HARNESS:` section proposing the automation that would make recurring findings impossible. They are used three ways: by CI on every PR (the review workflow), locally via `/review` before pushing, and as standing definitions of "good" that humans cite in disputes. When a persona repeats itself, that is a bug in the harness — encode the check and delete the repetition.

6.4 Commands and subagents

Slash commands package the rituals: `/plan-feature` (PRD → exec plan), `/implement-prd` (the full PM-friendly loop, stopping only at real decision points), `/qa-cuj` (Argent-driven QA session producing the standard report), `/sync-docs` (reconcile the tree with reality), `/review` (local persona pass). Subagents: `code-reviewer` (the persona matrix as one dispatchable reviewer) and `release-scribe` (writes the ship report and the Notion digest; honesty rule: *if the diff doesn't show it, it didn't ship*).

6.5 MCP: hands and instruments

`.mcp.json` registers **Argent** (`npx @swmansion/argent mcp`) — simulator/emulator control, accessibility-tree interactions, deep links, console/network logs, crash capture, React+native profiling — and **Notion** (official server, token via env). `.claude/settings.json` pre-allows the safe commands (verify, lint, test, expo) and denies the dangerous (reading `.env`, force-push, credentials), so day-one agent sessions are smooth *and* bounded.

7 · Part VI — CI/CD

7.1 The pipeline map

WORKFLOW	TRIGGER	WHAT IT DOES
<code>ci.yml</code>	every PR + main	npm ci → Prettier → ESLint (boundaries) → tsc → Jest+coverage → docs-lint → expo-doctor; coverage artifact
<code>claude-pr-review.yml</code>	PR open/update	Claude Code (your subscription via <code>CLAUDE_CODE_OAUTH_TOKEN</code>) adopts all five personas, posts one structured review with Verdict + HARNESS proposals; checks the PR template's proof-of-work; never auto-approves — humans keep merge authority
<code>ship-report.yml</code>	push to main	Claude reads the real commit range → writes <code>reports/ship-report.md</code> (engineering) → pandoc+WeasyPrint render a styled PDF artifact (90-day retention, also echoed to the job summary) → creates the Notion "What's new" page via Notion MCP under your parent page. No Claude secret? A deterministic fallback (<code>scripts/ship-report-fallback.mjs</code>) still groups Conventional Commits into a report
<code>e2e-ios.yml</code>	PR touching src/flows	macOS runner: prebuild → pod install → Release sim build → Maestro flows → JUnit + screenshots artifacts
<code>agentic-qa.yml</code>	nightly + manual	macOS runner builds the app, Claude Code + Argent MCP walks every CUJ with edge cases, screenshots anomalies, profiles jank, uploads the structured QA report (non-blocking by design; calibrate before trusting)
<code>deploy-preview.yml</code>	PR	EAS Update to a <code>pr-N</code> branch; QR-code comment so PMS/designers try the change on-device in minutes
<code>continuous-deploy.yml</code>	push to main	<code>continuous-deploy-fingerprint</code> : JS-only → OTA update to preview channel; native change → new EAS build. The fingerprint decides — humans don't
<code>release.yml</code>	tag <code>v*.*.*</code>	<code>eas build --platform all --profile production --auto-submit</code> + GitHub Release with generated notes

Plus `.eas/workflows/e2e-maestro.yml` — the same E2E expressed natively on EAS infrastructure if you prefer Expo's runners, and `renovate.json` tuned so Expo-coupled packages are *not* auto-bumped (SDK upgrades follow the runbook with `expo install --fix`).

7.2 The push → PDF → Notion automation, concretely

1. Merge to `main` (squash; Conventional title).
2. The workflow resolves the exact commit range and hands Claude Code the prompt at `.github/prompts/ship-report.md`.
3. Claude reads `git log` / `git show`, linked PRDs and exec plans; writes the four-section ship report (What shipped / Why it matters / Risk notes / Follow-ups).
4. pandoc converts the markdown to HTML with `.github/pdf/report.css`; WeasyPrint renders the PDF; both upload as an artifact named after the SHA.
5. The same session, via the Notion MCP, creates *"What's new — YYYY-MM-DD (sha)"* under `NOTION_PARENT_PAGE_ID` : 3–8 plain-language bullets for stakeholders, honest about internal plumbing.

Cost control: the job runs once per push to main, uses Sonnet by default, and skips to the deterministic fallback when no Claude credential is configured.

7.3 Testing strategy in one picture

Deterministic pyramid as the gate (Jest units → RNTL components → Maestro CUJ flows), agentic exploration as the net (nightly Argent sessions trying the things YAML never will: rapid double-taps, kill-mid-write, emoji titles, rotation). The two layers share one contract: **testIDs**. The QA persona blocks any PR that changes a CUJ without updating its flow.

8 · Part VII — The six-month adoption roadmap

Adapted from the OpenAI Frontier playbook to this repo. The skeleton front-loads most of "months 1–4" — your calendar time compresses, but the *sequence* still matters.

Months 1–2 — Make the repo legible (write things down)

- Replace every `TODO(company)` (identity, owners, vision); run the repo-setup runbook end-to-end; first trivial PR proves all automation.
- Migrate real tribal knowledge into `docs/`: how *your* team builds things, past architecture decisions as back-dated ADRs, your actual CUJs. Sunset knowledge stashed in Slack/Drive by moving or linking it.
- Norm: every "how do we...?" answered in chat gets written into docs the same day (`/sync-docs` helps).
- **Exit criteria:** an agent given a real ticket plans correctly from docs alone; docs-lint has caught ≥ 1 real drift; first feature shipped via `/implement-prd`.
- Expect this phase to feel slower than typing code. It is. That's the investment.

Months 3–4 — Encode taste into machinery

- Mine review comments and the personas' `HARNESS:` sections weekly; ship 1–3 new checks per week (lint rules, docs-lint extensions, tests). Targets you'll likely hit: import ordering, banned ad-hoc styling patterns, copy tone rules, performance smells (ScrollView+map), a11y minimums.
- Stand up the painted-door norm with design; first instrumented door ships.
- Trust the AI review enough to make human review intent-level: require persona P1s resolved *before* human eyes (already wired); measure how often humans repeat a persona.
- **Exit criteria:** a class of review comment you used to make weekly no longer occurs; agent PRs regularly arrive mergeable on first human read.

Months 5–6 — Open the system to non-engineers

- PMs ship via PRD + `/implement-prd` with engineers reviewing, not implementing. Designers ship token/primitive changes and painted doors directly. QA owns the nightly Argent calendar and its triage.
- Scale parallelism deliberately: more concurrent agent tasks per engineer ("five interns named Claude"), watching the safety metrics, not vibes.
- **Exit criteria:** ≥ 1 PM-shipped and ≥ 1 designer-shipped feature in production; engineers spend most time on triage, scheduling and harness work — and report it as an upgrade.

8.3 Metrics that tell you it's working

Throughput: merged PRs/week per operator; lead time PRD→production. Quality: % PRs mergeable without human-requested changes; escaped P1s from nightly QA; CUJ flake rate. **Flywheel (the most important):** harness checks added per week; persona findings per PR trending *down* while throughput trends *up*; share of feedback given once-then-encoded. Cost sanity: CI minutes and token spend per merged PR.

9 · Part VIII — Governance, security & failure modes

Authority. Humans hold merge authority — the AI review informs, never approves. ADRs gate architectural change; persona files and AGENTS.md changes go through the platform owners (CODEOWNERS).

Security posture. Nothing secret in the bundle (`EXPO_PUBLIC_*` is public by definition; Zod-validated env module is the only `process.env` reader). AsyncStorage is unencrypted — the security persona blocks tokens/PII until SecureStore lands. CI agents run least-privilege: review jobs read code and write comments; no store credentials ever reach an agent; `.claude/settings.json` denies `.env` reads and force-pushes locally. Renovate + weekly `npm audit` keep dependencies current without breaking SDK coupling.

Failure modes to expect (and the built-in answer): *agent writes plausible-but-wrong code* → closed loops: tests first, Argent attestation, persona review; *docs rot* → docs-lint fails the build; *agent fights a rule* → the rule wins by design — change it via ADR, not in-line; *spaghetti pressure under deadline* → boundaries are errors, painted door is the sanctioned shortcut; *automation theater* (checks nobody trusts) → the weekly harness retro prunes as well as adds; *runaway spend* → one ship-report per push, Sonnet default, fallback path, metrics reviewed monthly.

The cultural contract. Don't wrestle the agent in chat; fix the harness. Don't repeat feedback; encode it. Don't merge without proof; don't demand proof the machinery can provide automatically. Write things down — it remains the highest-leverage engineering work.

10 · Appendix

A. One-time setup checklist (full detail: docs/runbooks/repo-setup.md)

1. Replace `TODO(company)` marks (app identity in `app.config.ts`, CODEOWNERS teams, SECURITY contact, vision doc).
2. `eas init` + `eas update:configure` + credentials; paste projectId into `app.config.ts`.
3. GitHub: branch protection on main; secrets `EXPO_TOKEN`, `CLAUDE_CODE_OAUTH_TOKEN` (from `claude setup-token` — this is "use our Claude Code" in CI) or `ANTHROPIC_API_KEY`, `NOTION_TOKEN`; variable `NOTION_PARENT_PAGE_ID` (share that page with your Notion integration); install the Claude GitHub App.
4. Team machines: `npm install` (hooks self-install), `npx @swmansion/argent init`, Maestro CLI, optional Expo skills plugin for Claude Code.
5. Ship a trivial PR; verify every workflow fires; then onboard the team.

B. Cheatsheet

```
npm run verify          # everything CI checks, locally
npm run new:feature     # scaffold an on-architecture feature slice
/plan-feature ...      # PRD → exec plan          (Claude Code)
/implement-prd ...     # PRD → shipped PR          (Claude Code)
/review                # persona review pre-push  (Claude Code)
/qa-cuj               # Argent QA session        (Claude Code, macOS)
npm run e2e:ios        # Maestro flows locally
git tag v0.2.0 && git push --tags # store release
```

C. Sources consulted (June 2026)

- Expo SDK 56 changelog & docs — expo.dev/changelog/sdk-56 (RN 0.85, React 19.2, Hermes v1, AI scaffolding, router fork, precompiled modules)
- AGENTS.md — agents.md (open format; Agentic AI Foundation / Linux Foundation)
- anthropics/claude-code-action — github.com/anthropics/claude-code-action (OAuth token auth, prompt automation, MCP support)
- Argent — argent.swmansion.com · github.com/software-mansion/argent (simulator/emulator control, debugging, profiling; MCP + skills)
- Maestro — github.com/mobile-dev-inc/maestro; Expo E2E guide — docs.expo.dev/eas/workflows/examples/e2e-tests
- expo/expo-github-action — continuous-deploy-fingerprint, preview subactions
- EAS Workflows — docs.expo.dev/eas/workflows · fingerprint+repack CI acceleration — expo.dev/blog
- eslint-plugin-boundaries — github.com/javierbrea/eslint-plugin-boundaries
- NativeWind — nativewind.dev (v4 stable; v5 pre-release status)
- Zustand v5 / Zod v4 docs — [zustand](https://zustand.andrii.org) and [zod](https://zod.dev) official docs
- Notion API & official MCP server — developers.notion.com
- "Harness engineering" case study — Ryan Lopopolo (OpenAI Frontier), interview with Aakash Gupta, Jan 2026 (the article/transcript that motivated this handbook)

Built 2026-06-06 · Skeleton v0.1.0 · Expo SDK 56